

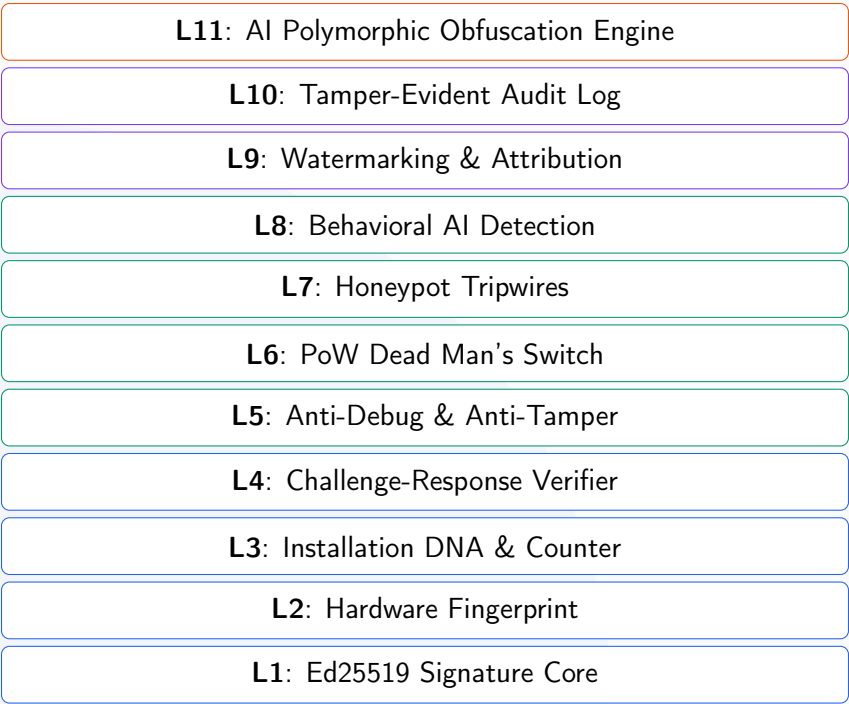
CertiGuard SDK

On-Premise Software License Protection

Defense-in-Depth: An 11-Layer Architecture

Prevention · Detection · Forensics · Offline-First · AI-Powered

AUP ESI Hackathon — April 2026



Contents

1	System Overview	2
1.1	The Problem	2
1.2	Why Existing Solutions Fail	2
1.3	CertiGuard’s Approach	2
2	Architecture Layers	2
2.1	Layered Design Diagram	3
2.2	Complete Layer Catalogue	3
3	Ed25519 Cryptographic Core	3
3.1	Mathematical Foundation	4
3.2	License File Structure	4
4	Installation DNA — Anti-Cloning	4
4.1	Three DNA Components	4
5	AI Behavioral Detection (L8)	4
5.1	Isolation Forest	5
6	Layer 11 — AI Polymorphic Obfuscation Engine	5
6.1	The Fundamental Problem with Existing Obfuscators	5
6.2	System Components	5
6.2.1	Subsystem A — Obfuscation Recipe Generator	6
6.2.2	Subsystem B — Custom VM ISA Generator (<code>vm_generator.py</code>)	7
6.2.3	Subsystem C — Dynamic VM-Check Noise Injector	8
6.3	The Graduated Tamper Response	9
6.4	Why a Reverser Who Cracked v1.0 Cannot Crack v1.1	10
6.5	CLI Integration	10
7	Complete Attack-Defense Matrix	12
8	SDK Integration	12
8.1	Minimal Integration Snippet	12
8.2	Deployment Scenarios	13
9	References	13

1 System Overview

1.1 The Problem

On-premise software is frequently attacked through license-file editing, debugger-assisted binary patching, virtual-machine cloning, and usage abuse. Traditional single-check licensing fails because attackers need to bypass only one gate. CertiGuard addresses this with a layered architecture where cryptographic trust, host binding, runtime verification, behavioral detection, and forensic evidence are chained into one offline-first control plane.

1.2 Why Existing Solutions Fail

Table 1: Limitations of common licensing approaches

Existing approach	Why it fails in practice
Plain serial key check	Static key checks are patched by flipping a branch instruction in minutes.
Online-only activation	Air-gapped or restricted enterprise networks cannot depend on real-time cloud checks.
Simple hardware lock	Single-factor hardware binding is bypassed by VM snapshots or partial spoofing.
Code obfuscation only	Deterministic obfuscation patterns are known to the cracking community; automated de-obfuscators exist for every major tool.
Periodic polling license server	“Kill server, keep app running” attack remains possible without local heartbeat enforcement.

1.3 CertiGuard’s Approach

CertiGuard implements a defense-in-depth model: signed license authenticity (L1), machine/installation context continuity (L2–L3), verifier challenge integrity (L4), anti-analysis and liveness controls (L5–L6), tamper tripwires (L7), local anomaly scoring (L8), leak attribution (L9), immutable audit chaining (L10), and a build-time AI polymorphic obfuscation engine (L11). The system remains operational offline while preserving post-event forensic visibility for vendor review.

2 Architecture Layers

2.1 Layered Design Diagram

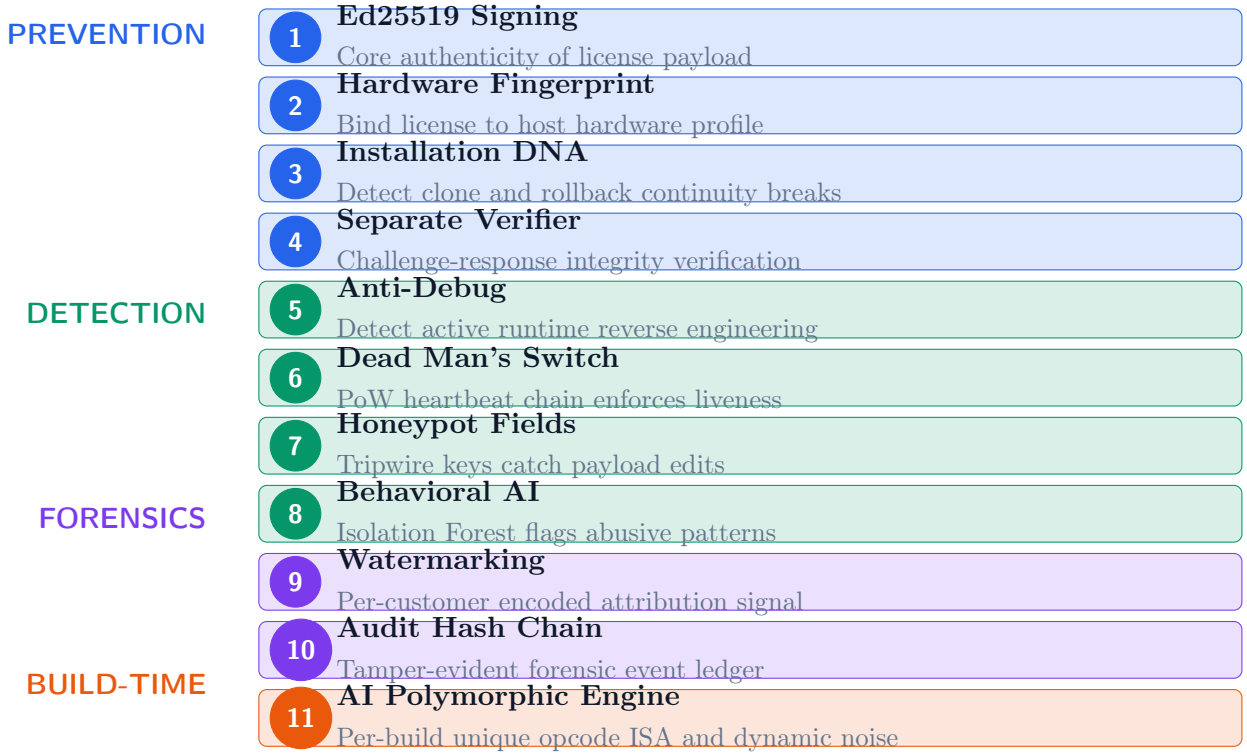


Figure 1: CertiGuard prevention-detection-forensics-build-time layer stack

2.2 Complete Layer Catalogue

Table 2: Layer model, threat focus, and offline suitability

Layer	Name	Threat Stopped	Algorithm/Method	Offline?
L1	Ed25519 Signing	File tampering, forgery	EdDSA on Curve25519	YES
L2	Hardware Fingerprint	License copying	SHA-256 + CPU/Board	YES
L3	Installation DNA	VM cloning, snapshots	UUID + Timestamp + Counter	YES
L4	Separate Verifier	Binary patching	Challenge-response HMAC	YES
L5	Anti-Debug	Debugger analysis	IsDebuggerPresent + timing + ptrace	YES
L6	Dead Man's Switch	Process killing	PoW heartbeat chain	YES
L7	Honeypot Fields	License editing	Tripwire fields in payload	YES
L8	Behavioral AI	Over-usage, sharing	Isolation Forest (sklearn)	YES
L9	Watermarking	Leak tracing	Per-customer encoding	YES
L10	Audit Log	Evidence destruction	SHA-256 hash chain	YES
L11	AI Polymorphic Engine	Static analysis, RE tools	Model-generated ISA + noise	YES

3 Ed25519 Cryptographic Core

3.1 Mathematical Foundation

CertiGuard's trust anchor is vendor-side Ed25519 signing. The client validates signatures using only the public key.

Private key: $d = \text{random_bytes}(32)$

Public key: $Q = d \cdot G$

Signature: $\sigma = (R, S)$, $S = (r + k \cdot d) \bmod L$

Verification: $S \cdot G = R + k \cdot Q$

Security Implication

Without the vendor private key d , a valid signature over a modified license payload is computationally infeasible under standard Ed25519 assumptions.

3.2 License File Structure

Listing 1: Conceptual signed license payload

```

1 {
2   "version": "3.0",
3   "license_id": "LIC-20260425025054",
4   "issued_to": "Customer A",
5   "valid_until": "2027-04-25T02:50:54Z",
6   "hardware_fingerprint": "sha256(...)",
7   "install_dna": {
8     "install_id": "uuid-v4",
9     "first_boot": "2026-04-01T10:00:00Z",
10    "boot_count_at_issue": 37
11  },
12  "parameters": { "max_users": 50, "modules": ["core", "analytics"] },
13  "watermark": "customer-bound-short-hash",
14  "tripwires": { "premium_unlock": false, "admin_override": false }
15 }
```

4 Installation DNA — Anti-Cloning

4.1 Three DNA Components

Table 3: DNA continuity checks under VM clone/snapshot conditions

Component	Original VM	Clone after snapshot	Result
UUID	a3f9...	Cannot decrypt (wrong HW)	REJECTED
First-Boot	2026-04-01	2026-04-15 (new boot)	MISMATCH
Boot Counter	150	100 (went backward)	REJECTED

5 AI Behavioral Detection (L8)

5.1 Isolation Forest

Isolation Forest isolates anomalous points using random partition trees; observations requiring fewer splits are more likely to be outliers.

Table 4: Behavioral features used for local anomaly scoring

Name	Normal Range	Fraud Indicator
Active users	5–60	Sudden spikes beyond licensed capacity
Hour-of-day pattern	Business-hour dominant	High activity at abnormal hours
Unique machines (24h)	Stable tenant profile	Sharp fan-out across many hosts
API calls/minute	Predictable envelope	Burst amplification and scripted cadence
CPU percent	10–75%	Sustained high load from automation loops
Memory percent	20–80%	Unusual sustained pressure with license abuse

6 Layer 11 — AI Polymorphic Obfuscation Engine

Core Innovation

Every existing obfuscation tool — ConfuserEx, Themida, OLLVM, Tigress — applies *deterministic* transformations. The cracking community has spent years reverse-engineering these patterns. Automated de-obfuscators exist for all of them. CertiGuard’s Layer 11 eliminates this attack surface entirely: a lightweight generative model produces a **unique, structurally novel obfuscation recipe and custom bytecode ISA on every single build**. A reverser who fully understands build v1.0 gains *zero* transferable knowledge about v1.1.

6.1 The Fundamental Problem with Existing Obfuscators

When a reverser opens a binary protected by ConfuserEx or OLLVM, they do not need to understand the obfuscation from scratch. They run a known de-obfuscation tool, or they recognise the control-flow patterns from previous experience, and they are back to reading near-original source code within minutes. The pattern is the weakness.

The insight behind Layer 11 is that **randomness at the transformation level, not just the parameter level**, is required to deny pattern-based reversal. This means the sequence of transformations applied, the instruction mappings chosen, the structure of the noise, and the layout of the VM interpreter must all differ between builds in a way that no static ruleset can predict.

6.2 System Components

Layer 11 comprises three integrated subsystems that operate at build time, before any binary is shipped to a client.

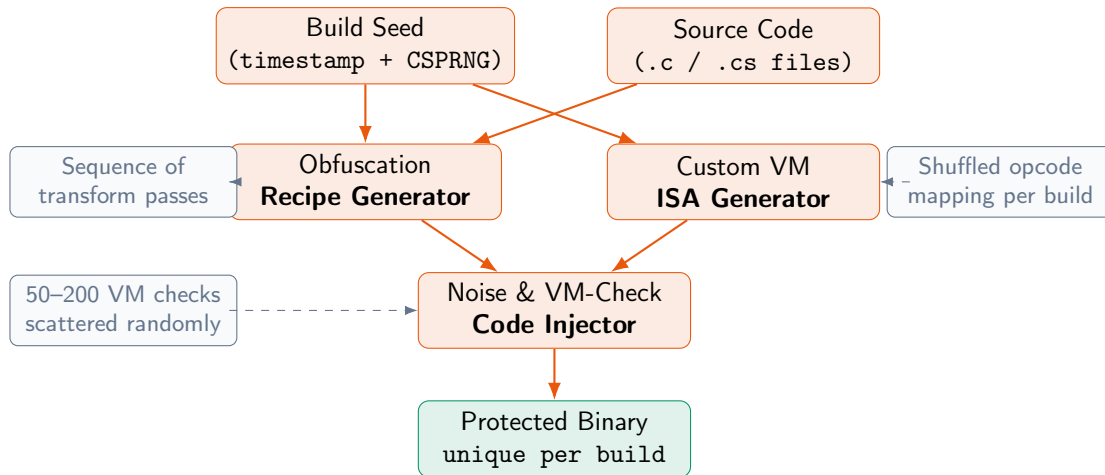


Figure 2: Layer 11 build-time pipeline: three subsystems combine to produce a unique binary per build

6.2.1 Subsystem A — Obfuscation Recipe Generator

A standard obfuscator applies a fixed pipeline of transformations. The Recipe Generator instead produces a **sequence** of transformation passes that is different on every build. The available transformation vocabulary includes:

- **Control-flow flattening** — replaces structured `if/else/loop` logic with a state-machine dispatcher
- **Opaque predicates** — inserts dead conditional branches whose outcome is mathematically fixed but not deducible by static analysis
- **Basic block splitting** — divides contiguous code regions into non-contiguous fragments connected by unconditional jumps
- **Arithmetic substitution** — replaces `x + y` with equivalent but complex expressions such as `(x ^ y) + 2*(x & y)`
- **Function cloning** — duplicates critical functions with structurally different but semantically identical implementations; call sites are routed randomly
- **String XOR encryption** — all string literals encrypted at compile time, decrypted at runtime with per-build keys
- **Bogus control flow** — dead code paths that appear plausible to a disassembler but are never executed

The generator draws a **random ordered subset** of these passes and random intensity parameters for each, producing a recipe object:

Listing 2: Illustrative recipe structure produced by the generator

```

1 # Build v1.0 recipe (seed 1001)
2 recipe_v1 = [
3     ("arithmetic_substitution", {"depth": 3}),
4     ("string_xor_encrypt", {"key_bits": 8}),
5     ("control_flow_flatten", {"dispatcher": "switch"}),
6     ("opaque_predicates", {"count": 12, "type": "polynomial"}),
7     ("function_clone", {"targets": ["verify_license", "
    check_hw"]}),

```

```

8 ]
9
10 # Build v1.1 recipe (seed 1002) -- entirely different ordering and
    params
11 recipe_v11 = [
12     ("basic_block_split",      {"granularity": "fine"}),
13     ("bogus_control_flow",     {"density": 0.4}),
14     ("function_clone",         {"targets": ["check_hw", "
        compare_fingerprint"]}),
15     ("string_xor_encrypt",      {"key_bits": 16}),
16     ("opaque_predicates",       {"count": 7, "type": "bitwise"}),
17     ("arithmetic_substitution", {"depth": 2}),
18 ]

```

A reverser who learned the pattern of v1.0 — which transformations were applied, in which order, with which parameters — will find that v1.1 presents a completely different structural signature. No de-obfuscator trained on v1.0 applies to v1.1.

6.2.2 Subsystem B — Custom VM ISA Generator (`vm_generator.py`)

The most protected code region — the license validation logic, hardware fingerprint comparison, and signature verification — is compiled to a **custom bytecode** and executed inside a generated C interpreter. Critically, the mapping from logical operations to physical opcode bytes is re-randomised on every build.

Table 5: Example opcode mapping across three builds — same logic, completely different byte representation

Logical operation	Build v1.0 (seed 42)	Build v1.1 (seed 43)	Build v1.2 (seed 44)
LOAD	0xA7	0x3C	0xF1
XOR	0x1B	0xD9	0x08
ADD	0x5E	0x74	0xB2
CMP	0xC3	0x2A	0x6D
JMP	0x9F	0x88	0x4E
EXIT	0x02	0xE5	0xCA
NOISE	0x6A-0x72	0x10-0x18	0x91-0x99

The bytecode stream for the license check also differs: noise bytes are injected at random positions between real instructions, and the XOR keys used in the validation arithmetic change per build. An attacker who mapped out the bytecode of v1.0 must restart from zero for v1.1.

The generated C header (`certiguard_vm.h`) contains both the opcode definitions and the interpreter loop, all produced freshly at build time by `vm_generator.py`:

Listing 3: Fragment of a generated VM interpreter — opcode values and bytecode are unique per build

```

1 /* CertiGuard Dynamic VM ISA -- Build Seed: 1001 */
2 #define VM_OP_LOAD  0xA7
3 #define VM_OP_XOR    0x1B
4 #define VM_OP_CMP    0xC3
5 #define VM_OP_EXIT   0x02
6
7 /* Bytecode for this build (noise bytes interspersed) */
8 static const unsigned char cg_bytecode[] = {
9     0x6A, 0xFF,          /* noise + operand -- skipped by interpreter
        */

```

```

10     0x1B, 0x1F,          /* XOR reg0 with build-specific key 0x1F
11         */
12     0xC3, 0x7E,          /* CMP reg0 against pre-computed target 0x7E
13         */
14     0x02                  /* EXIT -- return comparison result
15         */
16 };
17
18 static int certiguard_vm_execute(unsigned char input_val) {
19     int reg0 = input_val;
20     int pc = 0;
21     while (pc < sizeof(cg_bytecode)) {
22         unsigned char op = cg_bytecode[pc++];
23         if (op == VM_OP_XOR) { reg0 ^= cg_bytecode[pc++]; }
24         else if (op == VM_OP_CMP) { return (reg0 == cg_bytecode[pc++]); }
25         ; }
26         else if (op == VM_OP_EXIT) { return 0; }
27         else { pc++; } /* skip noise operand */
28     }
29     return 0;
30 }

```

What the attacker faces

To reverse-engineer the validation logic, the attacker must first reverse-engineer the VM interpreter to understand what each opcode does, then decode the bytecode stream, then strip out the noise bytes whose positions are unknown. For a conventional obfuscator, these steps are automated. For a per-build custom ISA, there is no tool — the attacker must perform this work from scratch for every single build.

6.2.3 Subsystem C — Dynamic VM-Check Noise Injector

Anti-VM and anti-debug checks are the most targeted code in any protected binary, because reversers know they must find and neutralise these checks to proceed. Conventional protection tools insert a small, known set of checks at predictable locations (startup, license load). CertiGuard's injector takes a fundamentally different approach.

At build time, between 50 and 200 VM-detection and integrity-check function stubs are **generated and scattered throughout the entire binary**. The number, positions, and implementation variants are all determined by the build seed:

Listing 4: Examples of generated noise check functions — different every build

```

1  /* Generated stubs -- positions, names, and implementations vary per
2     build */
3
4  void _cg_chk_47(void) {
5     /* Timing check variant B */
6     uint64_t t1 = __rdtsc();
7     uint32_t eax, ebx, ecx, edx;
8     __cpuid(0, eax, ebx, ecx, edx);
9     uint64_t t2 = __rdtsc();
10    if ((t2 - t1) > CG_TIMING_THRESHOLD_BUILD) cg_trigger_canary(0x4F);
11 }
12
13 void _cg_chk_112(void) {
14     /* CPUID vendor string check variant A */
15     __cpuid(0x40000000, eax, ebx, ecx, edx);

```

```

15     if (ecx == 0x4D566572) cg_trigger_canary(0x81); /* "rVeM" fragment
        */
16     /* Followed by arithmetic noise to confuse decompiler */
17     volatile int _pad = (eax * 17 + ecx) ^ 0xDEAD;
18     (void)_pad;
19 }
20
21 void _cg_chk_203(void) {
22     /* Registry artifact scan -- Windows only */
23     HKEY hk;
24     if (RegOpenKeyExA(HKEY_LOCAL_MACHINE,
25                     "SOFTWARE\\VMware, Inc.\\VMware Tools",
26                     0, KEY_READ, &hk) == ERROR_SUCCESS) {
27         RegCloseKey(hk);
28         cg_trigger_canary(0xC2);
29     }
30 }

```

These stubs are injected at random call sites throughout the existing codebase: inside loop bodies, after function prologues, inside rendering callbacks, inside save/load routines. The attacker cannot distinguish real checks from noise checks, cannot predict which ones affect execution, and cannot map them using knowledge of a previous build.

Table 6: VM/environment detection signals covered by the injected stubs

Signal Class	Detection Method	Spoofing Difficulty
CPUID hypervisor bit	ECX bit 31 on leaf 1	Easy alone
Hypervisor vendor string	CPUID leaf 0x40000000	Easy alone
Instruction timing	RDTSC delta around CPUID	Hard (physics limit)
Registry artifacts	VMware/VBox registry keys	Medium
Process artifacts	vmtoolsd.exe, VBoxService.exe	Medium
Device handle artifacts	\\.\VMCIDev, VBoxGuest	Medium
SMBIOS strings	DMI type 1/2 manufacturer fields	Medium
MAC prefix	NIC OUI for known VM vendors	Easy alone
WMI vs IOCTL mismatch	Cross-path serial validation	Hard
Thermal sensor uniformity	Variance across sensor readings	Hard
DRAM timing PUF	Physical memory cell variation	Very hard

Scoring, not hard-fail

No single check produces a hard rejection. Each fired signal contributes a weighted score to a global VM-confidence counter. Only when the cumulative score exceeds a configured threshold does the tamper response trigger. This design eliminates false positives from individual signals while remaining robust against partial spoofing. The threshold and weights are themselves embedded inside the custom VM bytecode and vary per build.

6.3 The Graduated Tamper Response

When Layer 11 (or any layer) determines that an attack is in progress, the response is deliberately **non-immediate and non-obvious**. An instant crash at the exact moment of detection tells the attacker precisely which check fired. CertiGuard instead uses a graduated response pipeline:

1. **Detection event** logged to L10 audit chain with full signal details

2. **Random delay** of 30–300 seconds before any visible change
3. **Silent state corruption**: non-critical internal values begin returning slightly wrong results
4. **Behavioral degradation**: UI delays increase, output accuracy decreases subtly
5. **Soft lock** after 5–15 minutes with a generic, non-informative error code
6. **Forensic dump** encrypted and written to disk; synced to vendor dashboard on next connection

The attacker observes degrading behavior but cannot determine what triggered it, when it triggered, or which check was responsible. This asymmetry — the attacker must understand the entire system to defeat it, while we only need to catch them once — is the core operational advantage.

6.4 Why a Reverser Who Cracked v1.0 Cannot Crack v1.1

Table 7: Knowledge transfer between builds: zero-knowledge property of Layer 11

Knowledge gained from v1.0	Applies to v1.1?	Reason
Opcode byte values in the custom VM	NO	ISA re-randomised by <code>vm_generator.py</code>
Bytecode stream positions and noise locations	NO	Seed changes; positions regenerated
Which function stubs are real vs noise	NO	Injection map regenerated per build
Obfuscation pass order and parameters	NO	Recipe regenerated by model
String XOR decryption keys	NO	Keys derived from build seed
Timing thresholds in VM checks	NO	Embedded inside regenerated bytecode
Control-flow graph structure	NO	Flattening pass regenerated
Ed25519 public key (L1)	YES	Key rotation is a separate vendor decision
Hardware fingerprint algorithm (L2)	YES	Algorithm is stable; only binding changes

The two rows marked YES are deliberate: the cryptographic core and hardware binding algorithm are stable by design. Their security does not depend on obscurity — they are secure because of mathematical hardness (Ed25519) and physical uniqueness (hardware identifiers). Layer 11 protects the *analysis surface*, not the cryptographic guarantees, which is the correct separation of concerns.

6.5 CLI Integration

Layer 11 is fully integrated into the CertiGuard build pipeline via `cli.py`:

Listing 5: Build pipeline commands invoking Layer 11

```

1 # Step 1: Generate polymorphic VM header (unique per build seed)
2 python cli.py generate-vm \
3   --out build/certiguard_vm.h \
4   --seed $(python -c "import time; print(int(time.time()))")

```

```
5
6 # Step 2: Generate dynamic noise header with AI-assisted mode
7 python cli.py generate-noise \
8     --seed $BUILD_SEED \
9     --out build/cg_noise.h \
10    --mode polymorphic \           # AI-assisted recipe generation
11    --lang c \
12    --lines 150                   # inject 150 noise check stubs
13
14 # Step 3: Obfuscate the core license validation C source
15 python cli.py obfuscate-source \
16    --input src/license_core.c \
17    --out build/license_core_obf.c \
18    --seed $BUILD_SEED \
19    --intensity 5                 # maximum transformation intensity
20
21 # Step 4: Compile and protect the binary
22 python cli.py protect \
23    --exe build/app_compiled \
24    --out-dir build/protected/
```

Automation Note

The build seed is derived from the current UNIX timestamp combined with a CSPRNG byte, ensuring that no two builds share a seed even if compiled within the same second. The seed is discarded after the build completes and is never embedded in the binary — making it impossible to reconstruct the original recipe from the output artifact.

7 Complete Attack-Defense Matrix

Table 8: Attack-to-layer mapping across 17 scenarios

ID	Attack Description	Layer	Detection Method	Result
A1	Edit signed field in license payload	L1	Ed25519 signature verification fails	Rejected
A2	Replace <code>valid_until</code> with future date	L1	Signature mismatch on modified payload	Rejected
A3	Copy license to a different machine	L2	Hardware fingerprint mismatch	Rejected
A4	Clone VM image to another host	L2/L3	HW mismatch + DNA continuity break	Rejected
A5	Snapshot rollback to older state	L3	Monotonic counter regression	Rejected
A6	Patch executable branch around verify	L4	Challenge-response integrity fails	Rejected
A7	Swap verifier response blob	L4	HMAC challenge mismatch	Rejected
A8	Attach debugger to runtime process	L5	Anti-debug detectors + timing heuristics	Blocked
A9	Single-step runtime to extract logic	L5	Timing anomaly and debugger flags	Blocked
A10	Kill watchdog/liveness process	L6	PoW heartbeat stale/chain invalid	Restricted
A11	Add <code>PREMIUM_UNLOCK=true</code> in payload	L7	Honeypot tripwire fires	Rejected
A12	Share one license across many teams	L8	Isolation Forest score over threshold	Flagged
A13	Leak customer license on forums	L9	Per-customer watermark attribution	Traced
A14	Delete/modify audit evidence rows	L10	Hash-chain verification failure	Alert
A15	Run app inside VM to study checks	L11	Multi-signal VM detector fires	Graduated response
A16	Use de-obfuscator on compiled binary	L11	Per-build ISA has no known de-obfuscator	No tool applies
A17	Apply crack from v1.0 to v1.1 build	L11	ISA, noise map and recipe all regenerated	Zero transfer

8 SDK Integration

8.1 Minimal Integration Snippet

Listing 6: Three-line host application integration

```

1 from certiguard.license_client import CertiGuardClient
2 result = CertiGuardClient(state_dir).verify_runtime(
3     license_path=lic, public_key_path=pub,
4     heartbeat_key=key, behavior_features=features)
5 assert result.ok, result.code

```

8.2 Deployment Scenarios

Table 9: Deployment modes and active layers

Scenario	Active Layers	Internet Required
Air-gapped enterprise	L1–L11 full stack, all offline-capable	NO
Standard enterprise	Full stack + opportunistic audit sync	NO
Trial / evaluation copy	L1 + L7 + L11 lightweight mode	NO

9 References

- [1] D. J. Bernstein et al., “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, 2011.
- [2] IETF, “RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA),” 2017.
- [3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” *IEEE ICDM*, 2008.
- [4] B. De Sutter, A. Slowinska, and S. Schrittwieser, “Software protection methodologies: State of the art and open challenges,” *ACM Computing Surveys*, 2024.
- [5] S. Chow, P. A. Eisen, H. Johnson, and P. C. van Oorschot, “White-box cryptography and implementation attacks,” *SAC*, 2002.
- [6] C. Collberg, C. Thomborson, and D. Low, “A taxonomy of obfuscating transformations,” Technical Report 148, University of Auckland, 1997.
- [7] B. Horne, L. Matheson, C. Sheehan, and R. Tarjan, “Dynamic self-checking techniques for improved tamper resistance,” *ACM DRM Workshop*, 2001.
- [8] National Security Agency, “TPM Use Cases and Deployment Guidance,” Technical Report, 2024.
- [9] K. Gruss et al., “Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript,” *DIMVA*, 2016. (DRAM PUF background)
- [10] Y. Oren et al., “The Spy in the Sandbox: Practical Cache Attacks in JavaScript,” *ACM CCS*, 2015.
- [11] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.

Layer 11 Design Summary

The AI Polymorphic Obfuscation Engine operates entirely at build time, introduces no runtime overhead beyond the custom VM interpreter, requires no internet connectivity, and produces binaries where every analysis artefact — opcode mapping, bytecode layout, noise positions, transform structure — is unique and non-transferable between builds. Combined with the cryptographic guarantees of L1–L10, it closes the final attack surface: static and dynamic reverse engineering of the protection logic itself.